

KOHLER Bathroom Designer

Prompts, Workflow & Build Documentation

An AI-assisted bathroom bundle recommender and true-to-scale room layout tool, built on KOHLER's real product catalog.

Nikhil Bhide · Satkaam

Repository: github.com/NikeBhide/NikhilBhideKohler

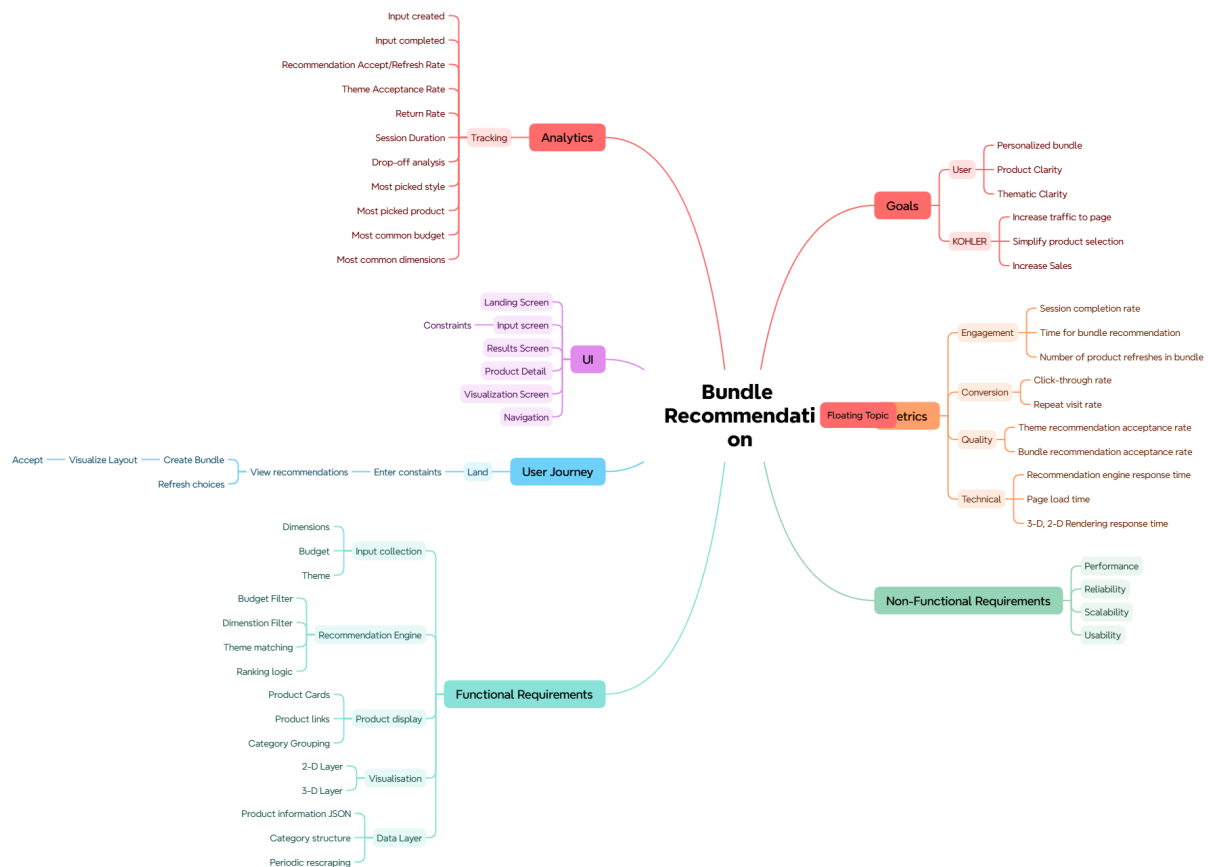
Built with Claude (Anthropic) as an AI development partner

Contents

- 1. Project Mind Map
- 2. Tech Stack
- 3. System Overview & Workflow
- 4. The Agents (What Each Piece Does)
- 5. Quality Testing & Validation Approach
- 6. AI-Assisted Build Process (Chronological)
- 7. Development Methodology & Skills Used
- 8. Future Scope

1. Project Mind Map

The planning mind map below scopes the full "Bundle Recommendation" product surface -- goals, functional requirements, UI constraints, the user journey, analytics/metrics, and non-functional requirements. The working model in this submission implements the core of Functional Requirements (input collection, recommendation engine, product display, 2D/3D visualization) and User Journey; Analytics/Metrics tracking and some UI screens are scoped for a later iteration (see Section 8, Future Scope).



Presented with xmind

Figure 1. Full product mind map (xmind).

2. Tech Stack

Layer	Choice	Why
Backend	Python 3 + Flask + Flask-CORS	Small, fast REST surface; no framework overhead for 5 endpoints
Frontend	React 18 + Vite	Fast dev server + HMR; no framework lock-in for a single-page guided flow
Styling	Plain CSS (per-component)	Full control over the isometric/plan SVG rendering; no CSS framework fighting custom layout math
“AI” layer	Deterministic weighted keyword/ pattern matching (no external LLM call)	Reproducible, auditable, zero external latency/cost; see Section 4 for how it still “reasons” about style
Data	Static JSON catalog (~1,500 real KOHLER SKUs, scraped)	No live-inventory dependency for a working prototype; swappable for a live feed later
Visualization	Inline SVG (isometric “3D” + plan view), hand-rolled projection math	No 3D engine dependency; renders straight from real placement coordinates, not a staged mockup
Testing	Custom Python test harnesses (unit-style + a 1,625-scenario sweep)	The placement engine's correctness (no overlaps, correct wall-hugging) is safety-critical to the UX
Dev tooling	Claude (Anthropic) as an AI pair-programming / debugging partner	See Sections 6 & 7 for how it was used

3. System Overview & Workflow

The system is a request/response pipeline, not a chat-with-an-LLM app: every “AI” decision (theme inference, product scoring, spatial placement) is a deterministic function over the real product catalog, which makes every recommendation reproducible and every layout decision explainable via the violations it reports.

End-to-end flow

- **Landing & setup** -- user picks product categories they want in the bundle.
- **Vision → Theme** -- user describes their bathroom in free text. *ExtractionAgent* scores it against 25 KOHLER design themes and always returns a confident best guess (never a dead end); the user can override the pick.
- **Budget & room size** -- budget (■) and room width/depth (inches) are collected.
- **Recommend** -- *RecommendationAgent* builds a cohesive, budget-bounded bundle from the real catalog and returns a plain-language rationale for why those products were chosen together.
- **Review & refine** -- user can lock products they like and refresh the rest; only unlocked categories are re-optimized.
- **Build the room** -- *PlacementAgent* lays the bundle out on a to-scale grid, respecting hard spatial rules, and reports any violation honestly instead of hiding it.
- **Visualize** -- the room renders as an isometric “3D” view and a plan view, both driven by the real placement coordinates; clicking a fixture opens its real KOHLER product page.

Request pipeline (per bundle build)

Frontend (React) → POST /api/recommend → **KohlerBathroomDesigner** (orchestrator) → **RecommendationAgent** (select bundle) → **PlacementAgent** (lay out bundle on grid) → JSON response (bundle + placement map + violations + rationale) → **GridVisualization** (2D/3D render).

4. The Agents (What Each Piece Does)

The backend is organized as four cooperating modules, each literally named “*_agent.py” in the codebase, plus an orchestrator that glues them together.

ExtractionAgent (backend/extraction_agent.py)

Infers a design theme from either a product's catalog description or a user's freeform “vision” text, by scoring it against 25 hardcoded KOHLER design themes using weighted keyword and inference-pattern matching (70% keyword-match ratio + 30% pattern match).

- Originally tuned for rich catalog copy; short casual user text (“cozy and warm, nothing too fancy”) often scored below the confidence threshold and returned nothing.
- Fixed by adding a `guarantee_result` mode: below-threshold input now returns the highest-scoring theme(s) anyway, flagged `low_confidence: true`, with a hardcoded `Transitional` fallback for text with no signal at all -- so the agent never dead-ends into “couldn't match, just pick one yourself.”

RecommendationAgent (backend/recommendation_agent.py)

Selects a cohesive, budget-bounded product bundle from the ~1,500-SKU catalog. Hard constraints: price $\leq$ budget, category in the user's selection, in stock. Soft scoring: 35% theme match, 50% bundle cohesion (sub-weighted 40% shared primary theme / 30% color / 20% price tier / 10% visual style), 15% category balance.

- Fixed a non-determinism bug where the same inputs produced different bundle sizes across identical requests -- caused by iterating a Python set of categories, whose order depends on interpreter hash seed.
- Fixed with a fixed `CATEGORY_PRIORITY_ORDER` list that makes category fill order (and therefore the result) reproducible.

PlacementAgent (backend/placement_agent.py)

Classifies every bundle item into a functional station (Wash / Toilet / Wet) and lays the stations out along the room's actual *longer* wall -- not always the back wall -- entrance-aware, in an abstract (along, away) coordinate space converted back to real (x, y) only once a position is accepted. Only 5 categories (Vanities, Basins, Toilets, Showers, Bathtubs) compete for floor space; everything else (faucets, mirrors, lighting, bidet seats, cabinets, accessories) is attached by coordinate math to its owning anchor.

- The single most-iterated module this build: the largest rewrite moved the assembly from always hugging the back wall to hugging whichever wall is actually longer, so a 60”×96” room runs the vanity/toilet/shower along its 96” wall instead of being forced into its 60” wall.
- Enforces hard rules: door-swing clearance kept clear, no anchor overlaps, required coincidences (faucet + mirror + light exactly on the vanity, bidet seat exactly on the toilet).
- Every capacity shortfall (room too small for the requested bundle) is reported as an explicit violation string, never silently dropped or allowed to overlap.

KohlerBathroomDesigner (backend/kohler_system.py)

The orchestrator: loads the real product catalog once, and on each request calls `RecommendationAgent` to build the bundle, then `PlacementAgent` to lay it out, then assembles the final API response (bundle, placement map, grid visualization, violations, primary theme).

- Exposes `infer_themes_from_text()` for the standalone vision-to-theme step, and `refresh_recommendation()` for the lock/refresh loop.

5. Quality Testing & Validation Approach

Because the placement engine's correctness is safety-critical to the user experience (an overlapping toilet and shower reads as a broken product, not a minor bug), it was the most heavily tested part of the system. Bugs were chased with a root-cause-first methodology rather than trial-and-error patching (see Section 7).

test_placement.py

8 hand-built room/bundle scenarios (spacious, typical, tight, no-vanity, no-toilet, shower-only, tub-only, minimal) asserting: faucet/mirror/light coincide with the wash anchor, bidet coincides with the toilet, the wash anchor hugs the assembly wall, no floor-anchor overlaps, entrance clearance is respected.

test_integration.py

4 full end-to-end scenarios run through the real recommendation + placement pipeline against the actual product catalog (not synthetic test products), checking the same coincidence/overlap rules on real KOHLER SKUs.

sweep2.py

A combinatorial sweep -- 13 room sizes × 5 theme sets × 5 category sets × 5 budgets = 1,625 scenarios -- checking for zero real overlaps, zero door-swing violations, and tracking how often the engine has to report “room is narrow/tight” or fails to place an item, across scales too large to hand-check.

Latest full-suite result (long-wall placement rewrite)

Check	Result
test_placement.py	8 / 8 scenarios fully passing
test_integration.py	4 / 4 scenarios fully passing
sweep2.py -- overlaps across 1,625 scenarios	0
sweep2.py -- door-swing violations across 1,625 scenarios	0
sweep2.py -- scenarios with an unplaced item	725 / 1,625 (down from a 775 baseline before this rewrite -- the long-wall fix improved packing, it didn't just relocate the bug)

Remaining unplaced-item cases are rooms genuinely too small for the requested bundle (e.g. a 60"×80" room asked to fit a vanity, toilet, shower, *and* a bathtub) -- reported honestly via violations rather than hidden or allowed to overlap.

6. AI-Assisted Build Process (Chronological)

This project was built in collaboration with Claude (Anthropic) acting as an AI pair-programmer: the author described the product, reported bugs and design rules in plain language (often from a live screenshot of the running app), and Claude investigated, designed, implemented, tested, and shipped the fix -- with the author reviewing behavior and correcting direction throughout. Below is a structured summary of that process, in the order it happened.

Phase 1 -- Initial build

Established the product shape via a structured brainstorming pass (see Section 7): a deterministic budget-bounded recommendation engine, a zoned functional-station placement layout (entrance / wash / toilet / wet), a wider non-scrollable colored-block room visualization with real product links (replacing an early ASCII-grid debug view), persisting the user's vision text across "Start Over," and a conversational "here's my thinking, want to change anything?" justification step before the bundle is finalized.

Phase 2 -- "It's still doing it" (fixtures in the wrong place)

The author reported fixtures appearing in the wrong place in the isometric 3D view. Root-cause investigation (systematic-debugging skill) found the placement zones were split 50/50 across the room width without regard to the wash anchor's actual footprint, starving the toilet/shower zones. Fixed with footprint-aware zone reservation; the assembly order was corrected from Toilet-Wash-Wet to Wash-Toilet-Wet after the author's explicit correction ("keep the shower at the end, the toilet in the middle"). A second bug -- a leftover boundary calculation letting wide shower/tub items creep back into the wash/toilet zone -- was found and fixed the same way.

Phase 3 -- Three-part bug report (theme dead-ends, wrong-wall layout, tiny 3D view)

The author sent one detailed message covering three issues, including a screenshot showing fixtures cascading diagonally in one corner of the room. Each was root-caused and fixed independently: (1) the theme agent's "sorry, couldn't match, just pick one yourself" dead end was replaced with an always-confident best-guess response; (2) the screenshot confirmed fixtures were being forced along the room's short wall -- the deepest fix of the project, described in Section 6, Phase 4; (3) the 3D view opening too small after a bundle is built was queued for investigation.

Phase 4 -- The long-wall rewrite (major architectural fix)

Rebuilt the placement engine's core around an abstract (along, away) coordinate space so the fixture assembly runs along whichever wall is actually longer, reflecting the axis so "entrance → wash → toilet → wet" reading order is preserved even when the assembly and the door end up on different walls. A hard frontend constraint (the door is always drawn at a fixed grid edge) ruled out a naive full-grid transpose, requiring this more surgical local-coordinate-space approach instead.

Phase 5 -- Verifying the rewrite (this session)

Running the existing test suite against the rewrite initially showed widespread failures. Root-cause analysis found the failures were in the *test harness itself* (it recomputed unrotated product footprints, which is wrong once items can be placed against a side wall) rather than the engine -- fixed by exposing the engine's own correctly-rotated rectangles for the tests to consume directly. A second, real bug was then found by manual inspection: a station whose zone was pre-computed before knowing where the previous station actually landed could end up overlapping it once the entrance-clearance carve-out pushed that station off its predicted spot. Fixed by making each station's zone slide to the previous station's *actual* landing spot rather than a static

prediction. Verified via the full test suite plus a 1,625-scenario sweep (Section 5) before shipping.

Phase 6 -- The 3D-view-too-small fix

Investigated the queued issue from Phase 3: the visualization panel's constraint-violations box and category legend had no height limit, so once a room reported a few "room is tight" notes (now common after Phase 4's honest-reporting behavior), they silently consumed most of the panel and crushed the actual 3D scene to a sliver. Fixed by capping those boxes (scrollable if needed), giving the 3D/plan view a hard minimum size and the majority share of the panel's height, and widening the panel itself.

Phase 7 -- Submission packaging (this document)

Cleaned the repository for submission (removing stale duplicate files, one-off scraper scripts, and raw scraped data dumps that predate the curated catalog), wrote fresh setup documentation, and produced this Prompts Documentation PDF and the accompanying presentation deck.

Representative user prompts (verbatim)

Quoted exactly as given, to show the kind of plain-language, sometimes informal bug reports and hard rules this build was steered by:

"keep the shower at the end. the toilet in the middle. ill make the dimenstions 80 into 120."

"...the agent is just saying sorry i couldn't match this to a style...it has to say that i can't just say sorry i couldn't match this to a theme and then move on to a theme that doesn't make any sense..."

"...everything is right next to each other horizontally, nothing is against the further wall...i have made hardbound rules that make sense...they have to be along the wall, along the long wall not the short wall, i made this a rule stop."

"...the bundle is opening in the 3d view but it's opening and it's so small on the screen, make it bigger, enhance it, these are basic things please."

7. Development Methodology & Skills Used

Two structured development skills (packaged, reusable playbooks that Claude follows when invoked) governed how this project was built and debugged, rather than ad hoc back-and-forth.

Brainstorming (product design before implementation)

Used at the start of the project to turn the initial idea into a concrete, agreed design before any code was written. Classifies the size of the change (a quick spike, a bounded fix, or full architectural work), asks clarifying questions one at a time, proposes options with trade-offs, and requires explicit approval of the design before implementation starts -- preventing the common failure mode of an AI assistant guessing at requirements and building the wrong thing.

Systematic Debugging (root cause before any fix)

The core rule: *no fix without a root-cause investigation first*. Every bug in Section 6 was chased through four phases -- reproduce and read the evidence, compare against a known-working case, form a single testable hypothesis, then implement and verify the fix -- rather than guessing at a plausible-looking patch. This is what caught, for example, that a batch of test failures after the long-wall rewrite was actually a bug in the *test* code, not the engine -- a wrong conclusion ("the rewrite is broken, revert it") that guessing would very likely have reached.

Verification discipline

Every backend change in this project was verified against the test suite (Section 5) before being shipped to the running application, and every file transfer to the developer's machine was independently checksum-verified (md5) after writing, with an automatic retry-and-reverify on the rare silent write failure -- so "it's fixed" was never asserted without evidence.

8. Future Scope

This is a working prototype, not a finished product -- here's what we'd actually build next.

- **More than one of the same fixture.** Right now the system assumes one toilet, one vanity, one shower. Real bathrooms don't always work that way -- a larger or shared bathroom might genuinely need two toilets, or two basins side by side. The placement engine's station model was built to handle exactly one of each, so this means teaching it to place repeats of a category without them colliding or breaking the wash/toilet/wet reading order.
- **A real agent, not a keyword matcher.** The current "style brain" is a deterministic scoring function -- fast and predictable, but it's pattern-matching, not understanding. The next version should be an actual reasoning agent that can genuinely read what someone describes and think about what it implies for their bathroom, instead of scoring their words against a fixed keyword list.
- **Show the real products, not boxes.** The 3D view today draws every fixture as a flat colored cuboid -- functionally correct, but it doesn't look like a bathroom. The goal is to render the actual KOHLER products as real 3D models in the space, and let someone drag, rotate, and look around the room instead of viewing a fixed isometric angle.

Appendix: Development Skills Reference

The following are the complete skill definitions (reusable playbooks) that governed development, debugging, and verification. Each is a structured process that ensures discipline over ad hoc decisions.

Brainstorming: Product design before implementation

Brainstorming Ideas Into Designs

Help turn ideas into fully formed designs and specs through natural collaborative dialogue.

Start by classifying how much process the request needs, then work through your path: understand the context, refine the idea, present a design, and get your human partner's approval.

Establish Shared Understanding

The outcome of brainstorming is an understanding your human partner can recognize and correct, grounded in what they want to accomplish.

1. **Discover intent.** Use the request and available context to identify the intended outcome, who it is for, and what success looks like. When that information is missing, ask one focused question about purpose or intended use before proposing features or an approach. Knowing the app genre does not tell you why your partner wants it. Gathering missing requirements does not ask them to authorize the task again.
2. **Write back your understanding.** Summarize the intended outcome, relevant constraints, and success criteria in a short note your partner can assess. Separate what they said from assumptions. Invite correction and incorporate their answer before treating this as the design brief.
3. **Carry intent into the design.** Preserve the agreed understanding in the selected path's design artifact: the written spec for architectural work, or the in-chat design/probe for bounded work and spikes. Check proposed features and technical choices against that understanding.

When the request already supplies the purpose and constraints, reflect that understanding instead of asking the same questions again. Keep the note concise; its accuracy and the opportunity to correct it matter.

Before taking any implementation action, including invoking an implementation skill, writing product code, scaffolding, installing product dependencies, or creating an external project, complete the selected path's prerequisites:

- Spike: the human partner approves the question and probe.
- Bounded: the human partner approves the short in-chat design.
- Architectural: the human partner reviews and approves the written spec, then reviews the written implementation plan and selects its execution method. Conversational design approval only permits writing the spec; written-spec approval only permits invoking writing-plans.

A reply approves the stage actually presented. Approval of an idea or feature scope does not approve artifacts that do not exist yet. Resume at the earliest incomplete stage; do not turn one approval into permission to skip the rest of the selected path. Read-only project exploration is allowed while those prerequisites remain incomplete.

Three Paths

Before your first question, classify the request and say the classification out loud — "this looks bounded, so I'll present a short design here rather than write a spec" — so your human partner can override it:

- ****Spike**** — a feasibility question ("can we...", "is it possible...", "quick and dirty is fine") whose output is an answer, not code you keep. Present the question and what you'll try in 2-3 sentences, get a nod, then find out as cheaply as correctness allows. No design doc, no spec file. Report findings as a recommendation; anything you built stays labeled throwaway.
- ****Bounded**** — a well-scoped change to code that already exists in this repo: a new flag, a small endpoint, a one-file fix.

Understanding the kind of app is not enough — bounded means the flow you are changing is already here to read. If there is no existing flow to change, the task is not bounded. Ask the clarifying

questions that matter, present a short design IN CHAT (a few sentences to a few short paragraphs), and STOP. Implementation starts only after your human partner says yes to that design — a bounded task's approval is as hard a gate as an architectural one. No spec file, no implementation plan document.

- ****Architectural**** — new projects, new subsystems, changes that restructure how components fit together or alter interfaces others depend on. Follow the full process: questions, approaches, sectioned design, written spec, then the writing-plans skill.

Systematic Debugging: Root cause before any fix

Systematic Debugging

Overview

Core principle: ALWAYS find root cause before attempting fixes. Symptom fixes are failure.

Violating the letter of this process is violating the spirit of debugging.

The Iron Law

...

NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST

...

If you haven't completed Phase 1, you cannot propose fixes.

When to Use

Use for ANY technical issue:

- Test failures
- Bugs in production
- Unexpected behavior
- Performance problems
- Build failures
- Integration issues

Use this ESPECIALLY when:

- Under time pressure (emergencies make guessing tempting)
- "Just one quick fix" seems obvious
- You've already tried multiple fixes
- Previous fix didn't work
- You don't fully understand the issue

Don't skip when:

- Issue seems simple (simple bugs have root causes too)
- You're in a hurry (rushing guarantees rework)
- Manager wants it fixed NOW (systematic is faster than thrashing)

The Four Phases

You MUST complete each phase before proceeding to the next.

Phase 1: Root Cause Investigation

****BEFORE attempting ANY fix:****

1. ****Read Error Messages Carefully****

- Don't skip past errors or warnings
- They often contain the exact solution
- Read stack traces completely
- Note line numbers, file paths, error codes

2. ****Reproduce Consistently****

- Can you trigger it reliably?
- What are the exact steps?
- Does it happen every time?
- If not reproducible → gather more data, don't guess

3. ****Check Recent Changes****

- What changed that could cause this?
- Git diff, recent commits
- New dependencies, config changes
- Environmental differences

4. ****Gather Evidence in Multi-Component Systems****

****WHEN system has multiple components (CI → build → signing, API → service → database):****

****BEFORE proposing fixes, add diagnostic instrumentation:****

...

For EACH component boundary:

- Log what data enters component
- Log what data exits component
- Verify environment/config propagation
- Check state at each layer

Run once to gather evidence showing WHERE it breaks

THEN analyze evidence to identify failing component

THEN investigate that specific component

...

Test-Driven Development: Tests drive implementation

Test-Driven Development (TDD)

Overview

Write the test first. Watch it fail. Write minimal code to pass.

Core principle: If you didn't watch the test fail, you don't know if it tests the right thing.

Violating the letter of the rules is violating the spirit of the rules.

When to Use

Always:

- New features
- Bug fixes
- Refactoring
- Behavior changes

Exceptions (ask your human partner):

- Throwaway prototypes
- Generated code
- Configuration files

Thinking "skip TDD just this once"? Stop. That's rationalization.

The Iron Law

...

NO PRODUCTION CODE WITHOUT A FAILING TEST FIRST

...

Write code before the test? Delete it. Start over.

No exceptions:

- Don't keep it as "reference"
- Don't "adapt" it while writing tests
- Don't look at it
- Delete means delete

Implement fresh from tests. Period.

Red-Green-Refactor

```
``dot
digraph tdd_cycle {
rankdir=LR;
red [label="RED\nWrite failing test", shape=box, style=filled, fillcolor="#ffcccc"];
verify_red [label="Verify fails\nincorrectly", shape=diamond];
green [label="GREEN\nMinimal code", shape=box, style=filled, fillcolor="#ccffcc"];
verify_green [label="Verify passes\nAll green", shape=diamond];
refactor [label="REFACTOR\nClean up", shape=box, style=filled, fillcolor="#ccccff"];
next [label="Next", shape=ellipse];

red -> verify_red;
verify_red -> green [label="yes"];
verify_red -> red [label="wrong\nfailure"];
green -> verify_green;
verify_green -> refactor [label="yes"];
verify_green -> green [label="no"];
refactor -> verify_green [label="stay\ngreen"];
verify_green -> next;
next -> red;
}
...

```

RED - Write Failing Test

Write one minimal test showing what should happen.

```
``typescript
test('retries failed operations 3 times', async () => {
let attempts = 0;
const operation = () => {
attempts++;
if (attempts < 3) throw new Error('fail');
return 'success';
}
}

```

```
};
```

```
const result = await retryOperation(operation);
```

Verification Before Completion: Verify every claim

Verification Before Completion

Overview

Core principle: Evidence before claims, always.

Violating the letter of this rule is violating the spirit of this rule.

The Iron Law

NO COMPLETION CLAIMS WITHOUT FRESH VERIFICATION EVIDENCE

If you haven't run the verification command in this message, you cannot claim it passes.

The Gate Function

BEFORE claiming any status or expressing satisfaction:

1. IDENTIFY: What command proves this claim?
2. RUN: Execute the FULL command (fresh, complete)
3. READ: Full output, check exit code, count failures
4. VERIFY: Does output confirm the claim?
 - If NO: State actual status with evidence
 - If YES: State claim WITH evidence
5. ONLY THEN: Make the claim

Skip any step = lying, not verifying

Common Failures

| Claim | Requires | Not Sufficient |

|-----|-----|-----|

| Tests pass | Test command output: 0 failures | Previous run, "should pass" |

| Linter clean | Linter output: 0 errors | Partial check, extrapolation |

| Build succeeds | Build command: exit 0 | Linter passing, logs look good |

Bug fixed	Test original symptom: passes	Code changed, assumed fixed
Regression test works	Red-green cycle verified	Test passes once
Agent completed	VCS diff shows changes	Agent reports "success"
Requirements met	Line-by-line checklist	Tests passing

Red Flags - STOP

- Using "should", "probably", "seems to"
- Expressing satisfaction before verification ("Great!", "Perfect!", "Done!", etc.)
- About to commit/push/PR without verification
- Trusting agent success reports
- Relying on partial verification
- Thinking "just this once"
- Tired and wanting work over
- ****ANY wording implying success without having run verification****

Rationalization Prevention

Excuse	Reality
"Should work now"	RUN the verification
"I'm confident"	Confidence $\neq$ evidence
"Just this once"	No exceptions
"Linter passed"	Linter $\neq$ compiler
"Agent said success"	Verify independently
"I'm tired"	Exhaustion $\neq$ excuse
"Partial check is enough"	Partial proves nothing
"Different words so rule doesn't apply"	Spirit over letter

Key Patterns

****Tests:****

...

■ [Run test command] [See: 34/34 pass] "All tests pass"

■ "Should pass now" / "Looks correct"

...

****Regression tests (TDD Red-Green):****

...

■ Write → Run (pass) → Revert fix → Run (MUST FAIL) → Restore → Run (pass)

■ "I've written a regression test" (without red-green verification)